CONNECT 4

Name : Adeet Singh

Reg. No. : 25BAI11451

Course : BTECH CSE (AI/ML)

1. ABSTRACT

The project "First to 4" is a comprehensive exploration into the field of Artificial Intelligence (AI) and heuristic-based decision-making, implemented through the digital recreation of the classic Connect Four tabletop game. At its core, the project addresses the challenge of simulating a zero-sum game of perfect information within a terminal-based environment using the Python programming language.

The development process was bifurcated into two primary objectives: first, the construction of a robust game engine capable of enforcing physical constraints—such as gravity-fed piece placement and multi-directional win detection—and second, the engineering of a sophisticated adversary. The intelligence of the system is categorized into three distinct tiers, ranging from a stochastic "Easy" mode to a "Hard" mode powered by the **Minimax Algorithm** enhanced with **Alpha-Beta Pruning**.

Technical implementation focused heavily on algorithmic efficiency. To manage the vast state space of Connect Four, which consists of over 4.5 trillion possible board positions, a recursive search tree was employed. By implementing Alpha-Beta pruning, the system was optimized to discard mathematically irrelevant branches of the game tree, significantly reducing computational latency while maintaining a high level of strategic play.

The final result is a highly performant application that demonstrates the practical application of discrete mathematics and recursive logic. The project successfully bridges the gap between basic procedural programming and advanced algorithmic strategy, providing a scalable framework for further research into machine learning and automated game-solving.

---

2. INTRODUCTION

2.1 The Legacy of Connect Four

Connect Four, a game originally published by Milton Bradley in 1974, has long been a staple of combinatorial game theory. While it appears simple on the surface—requiring only the alignment of four discs—it is a "zero-sum game of perfect information." This means that there is no element of chance, such as dice or shuffled cards, and both players

have total visibility of the board state at all times. In 1988, James Allen and Victor Allis independently proved that the game is "mathematically solved," meaning that with perfect play, the first player can always force a win. This project seeks to replicate that level of strategic depth through a digital medium.

2.2 Objective and Motivation

The primary motivation behind this project was to explore the intersection of classical game mechanics and modern computational logic. Building a "First to 4" game in Python serves as a rigorous exercise in managing two-dimensional data structures and implementing recursive algorithms. The objective was not merely to create a playable game, but to engineer an intelligent system that could provide a scalable challenge to human players.

The project aims to demonstrate:

- **Physics Simulation:** Replicating the "gravity-fed" mechanic where pieces must stack vertically.
- **Heuristic Logic:** Creating an AI that can "value" a board position based on strategic advantages rather than just immediate wins.
- **Optimized Searching:** Implementing a search tree that can look several turns into the future without causing significant processing delays.

2.3 Scope of the Project

This application is designed as a standalone, terminal-based software package. While many modern games rely on high-end graphics, this project prioritizes **algorithmic integrity** and **logic efficiency**. By utilizing Python’s standard library, the project remains highly portable and serves as a blueprint for how complex decision-making processes can be coded from scratch. The scope includes a fully functional game loop, an automated win-detection engine, and a multi-tiered Artificial Intelligence module that transitions from random movement to advanced strategic planning.

---

3. PROBLEM ANALYSIS & REQUIREMENTS

3.1 Problem Definition

The primary challenge of developing a digital "First to 4" game lies in translating physical board mechanics into logical code structures. Unlike

a standard grid-based game like Tic-Tac-Toe, where any empty cell is a valid target, Connect Four introduces the element of **linear gravity**. This constraint dictates that a player's choice is limited to a column, while the specific row is determined by the existing state of that column.

From a computational standpoint, the problem can be categorized as a **State-Space Search** problem. With a 6×7 grid, the number of potential board configurations is approximately 4.53×10^{12}. Analyzing this vast number of possibilities in real-time requires a move-selection strategy that balances depth of search with processing speed.

3.2 Functional Requirements (FR)

Functional requirements define the core behaviors the system must exhibit for the game to be considered "complete."

- **FR1: Column-Based Input:** The system must accept an integer input (0-6) from the user and validate that the chosen column is not already at maximum capacity.
- **FR2: Gravity Simulation:** Upon selecting a valid column, the system must automatically calculate the lowest available row index and update the state of that cell.
- **FR3: Win Condition Evaluation:** After every move, the system must scan the board for four identical tokens aligned horizontally, vertically, or diagonally.
- **FR4: Difficulty Scaling:** The application must offer three distinct logic levels (Easy, Medium, Hard) to ensure the game remains accessible to beginners and challenging for experts.
- **FR5: Game State Termination:** The system must recognize "End-of-Life" states, such as a victory or a stalemate (full board with no winner), and prevent further inputs.

3.3 Non-Functional Requirements (NFR)

Non-functional requirements specify the "quality" attributes of the software.

- **NFR1: Performance/Latency:** Even on the "Hard" difficulty, the AI must return a decision within 2 seconds. A delay longer than this disrupts the user experience and breaks the flow of gameplay.

- **NFR2: Portability:** The software must be written in pure Python 3.x, utilizing only built-in modules to ensure it runs across Windows, macOS, and Linux without external dependencies.
- **NFR3: Error Handling:** The program must gracefully handle "dirty" inputs, such as letters or symbols, without crashing the execution loop.
- **NFR4: Visual Clarity:** The terminal interface must use distinct colors (ANSI escape codes) to separate the human player from the machine, facilitating immediate pattern recognition.

3.4 Constraint Analysis

The design of the "Hard" AI introduces a "Computational Budget" constraint. In a recursive search like Minimax, every additional level of depth increases the number of nodes evaluated exponentially (7^d). Therefore, a critical requirement was the implementation of a **Heuristic Evaluation Function**. Since the computer cannot "see" the end of every possible game branch, it must be able to assign a mathematical value to a non-terminal board state—essentially "guessing" how strong a position is based on factors like center-column control and open-ended sequences of three.

---

4. SYSTEM DESIGN & ARCHITECTURE

4.1 Architectural Pattern: Simplified MVC

The "First to 4" system is designed using a modified **Model-View-Controller (MVC)** architectural pattern. This separation of concerns ensures that the logic governing the game rules is decoupled from the user interface and the decision-making engine.

- **The Model (Data Layer):** This component manages the state of the game board. It consists of a 6×7 two-dimensional list. The Model is responsible for identifying if a move is legal and maintaining the "source of truth" for piece positions.
- **The View (Presentation Layer):** The View handles the rendering of the board to the terminal. By isolating the print_board function, the system can be upgraded to a Graphical User Interface (GUI) in the future without modifying the underlying game rules.
- **The Controller (Logic Layer):** This is the "Game Loop" that coordinates the human player’s input and the AI’s calculations. It

updates the Model based on inputs and triggers the View to refresh the display.

4.2 Data Structures and Representation

The selection of data structures was a critical design decision. A **2D List (Matrix)** was chosen over a 1D list or a dictionary for several reasons:

1. **Coordinate Mapping:** It allows for intuitive indexing using board[row][column], which matches the Cartesian-style grid of a physical board.
2. **Directional Iteration:** Checking for wins requires stepping through adjacent cells. A 2D array allows for simple nested loops to check horizontal, vertical, and diagonal neighbors using index offsets (e.g., row + 1, col + 1).
3. **Memory Efficiency:** A 6×7 matrix is extremely lightweight, occupying negligible RAM while providing $O(1)$ access time for any specific cell.

4.3 Component Diagram & Module Interaction

The system is divided into four primary modules that interact in a linear, synchronous flow:

1. **Initialization Module:** Sets up the empty grid and randomly determines which player (Human or AI) takes the first turn to ensure fairness.
2. **Validation Module:** Intercepts player input. It checks for two failure states: **Type Errors** (entering a non-integer) and **Logic Errors** (attempting to drop a piece into a column that is already full).
3. **The Drop Engine:** Simulates the physics of the game. It scans a column from index 5 (bottom) to 0 (top) to find the first available empty string, then updates that coordinate with the current player's token ('X' or 'O').
4. **The Evaluation Engine (AI):** If it is the AI's turn, the Controller passes a **Deep Copy** of the board to the AI module. The AI evaluates potential futures and returns the optimal column index to the Controller.

4.4 Simulation Strategy: Deep Copying

A major architectural hurdle was preventing the AI from "playing" its imagined moves on the actual game board. To solve this, the architecture employs a **Simulation-via-Cloning** strategy.

In Python, lists are passed by reference, meaning any change to a "copy" of a list usually affects the original. To bypass this, the copy.deepcopy() function is used. This creates a completely separate instance of the board in the system's memory. The AI can then simulate thousands of hypothetical moves on these "ghost boards" to calculate scores without interfering with the live game state visible to the human player.

---

5. ALGORITHM IMPLEMENTATION

5.1 The Gravity-Drop Algorithm

The most fundamental algorithm in the "First to 4" system is the piece placement logic, which simulates physical gravity. Unlike a standard X, Y coordinate selection, the user only provides the X (column) coordinate. The algorithm must then solve for the Y (row) coordinate.

The Implementation Process:

1. **Linear Scan:** The algorithm initiates a for loop starting from the maximum row index (Row 5, the bottom) and iterates upward to Row 0.
2. **Occupancy Check:** For each row, it checks if the cell at board[row][col] contains the EMPTY constant.
3. **Assignment:** Upon finding the first empty cell, the player's piece is assigned to those coordinates, and the function returns the row index to the caller.
4. **Overflow Handling:** If the loop completes without finding an empty cell, the function returns a None value, signaling to the controller that the column is full and the move is invalid.

5.2 Multi-Directional Win Detection

Detecting a win in Connect Four is significantly more complex than in Tic-Tac-Toe due to the larger grid and four possible win orientations. The system utilizes a **Sliding Window Algorithm** to verify victory states after every move.

5.2.1 Horizontal and Vertical Scanning

The horizontal scan iterates through every row and checks a "window" of four columns. For a row of 7 columns, there are 4 possible horizontal windows per row ($7 - 3 = 4$).

Similarly, the vertical scan iterates through every column and checks a window of four rows. For 6 rows, there are 3 possible vertical windows per column ($6 - 3 = 3$).

5.2.2 Diagonal Scanning Logic

Diagonal detection is the most mathematically demanding portion of the implementation. The algorithm must account for two types of slopes:

- **Positively Sloped Diagonals:** Moving from bottom-left to top-right.
- **Negatively Sloped Diagonals:** Moving from top-left to bottom-right.

The implementation uses nested loops with coordinate offsets. For any cell (r, c), a diagonal win is checked by verifying the state of $(r+1, c+1)$, $(r+2, c+2)$, and $(r+3, c+3)$. To prevent "Index Out of Range" errors, the loops are constrained to start only in positions where a 4-length sequence is physically possible within the 6×7 bounds.

5.3 The Heuristic Evaluation Function

For the AI to "understand" the board without seeing the end of the game, an evaluation algorithm was implemented to assign a numerical score to any given board state. This is essentially a **Pattern Recognition Algorithm**.

The Scoring Heuristic:

- **Window Scoring:** The algorithm breaks the entire board into every possible 4-slot "window" (horizontal, vertical, and diagonal).
- **Sequence Weighting:** * A window with 4 AI pieces = +1,000,000 (Winning move).
    - A window with 3 AI pieces and 1 empty slot = +5 (Strong offensive position).
    - A window with 2 AI pieces and 2 empty slots = +2 (Developmental position).

- A window with 3 Player pieces and 1 empty slot = -80 (Critical defensive block required).

5.4 Center Column Preference Algorithm

Strategic analysis of Connect Four shows that the center column (Column 3) is involved in the highest number of possible winning combinations. To reflect this, a specific algorithm was implemented to count the number of AI pieces in the center column and multiply them by a "Preference Factor" of 3. This encourages the AI to control the middle of the board early in the game, mirroring professional human strategy.

---

6. AI LOGIC & THE MINIMAX THEORY

6.1 The Philosophy of Adversarial Search

At the heart of the "Hard" difficulty setting is the **Minimax Algorithm**, a recursive decision-making strategy used in two-player, zero-sum games. In Connect Four, "zero-sum" implies that any advantage gained by the AI is a mathematical disadvantage for the human player. The goal of the AI is not merely to find a winning move, but to play "optimally"—assuming that the opponent will also play optimally.

The algorithm functions by building a **Game Tree**. Each node in the tree represents a possible board state, and each edge represents a legal move. The AI (the Maximizer) seeks to reach a state with the highest possible score, while the human (the Minimizer) is assumed to be trying to force the game into a state with the lowest possible score for the AI.

6.2 Recursive Depth and Terminal States

Because Connect Four has billions of possible states, the AI cannot calculate every move until the very end of the game (the "Leaf Nodes"). Instead, the implementation uses **Depth-Limited Minimax**.

1. **Depth Selection:** The system is configured to look 5 moves ahead. This "look-ahead" depth was chosen as a balance between strategic foresight and computational speed.
2. **Base Case:** The recursion stops if the game ends (Win/Loss/Draw) or if the maximum depth is reached.
3. **Backtracking:** Once the maximum depth is reached, the "Heuristic Evaluation Function" assigns a score to that board, and that value is passed back up the tree to help the AI decide its current move.

6.3 Optimization: Alpha-Beta Pruning

A standard Minimax search at a depth of 5 would require evaluating 7^5 (16,807) board positions every turn. To optimize this, **Alpha-Beta Pruning** was implemented. This logic allows the AI to "ignore" branches of the tree that cannot possibly influence the final decision.

- **Alpha (α):** The best value that the Maximizer (AI) can currently guarantee.
- **Beta (β):** The best value that the Minimizer (Player) can currently guarantee.

If at any point the algorithm finds a move where $\beta \le \alpha$, it "prunes" the rest of that branch. It mathematically proves that the player would never allow the game to reach that state, so there is no need for the computer to spend time calculating it. This optimization often reduces the number of nodes evaluated by over 50-70%, making the AI feel "instant" to the player.

6.4 The Heuristic Scoring System (The "Brain")

Since the AI often stops searching before a win is confirmed, it needs a way to judge if a board is "good" or "bad." This is handled by a scoring heuristic that evaluates every 4-slot "window" on the grid:

Pattern	Score	Strategic Logic
[AI, AI, AI, AI]	+1,000,000	Immediate victory; highest priority.
[AI, AI, AI, Empty]	+5	Creating a "threat" that requires a block.
[AI, AI, Empty, Empty]	+2	Early game development and space control.
[Player, Player, Player, Empty]	-80	High-priority defensive block to prevent loss.
Center Column Piece	+3	Controlling the middle to maximize win paths.

6.5 Handling the "Horizon Effect"

A common issue in game AI is the "Horizon Effect," where the AI makes a move that delays an inevitable loss because it cannot see past its

maximum depth. By prioritizing the center column and assigning a heavy penalty to the player's 3-in-a-row sequences, our implementation mitigates this, ensuring the AI plays defensively even when a clear win isn't visible within the 5-move window.

---

7. TESTING & QUALITY ASSURANCE

7.1 Validation Testing (Input Integrity)

The first layer of quality assurance involved **Stress Testing** the user input mechanism. Since terminal applications are prone to crashing if a user enters unexpected data types, a rigorous validation suite was implemented.

- **Type Checking:** We tested the system's response to non-integer inputs (e.g., strings, special characters, and empty returns). The implementation uses a try-except block to catch ValueError exceptions, ensuring the game loop continues rather than terminating.
- **Boundary Analysis:** Tests were conducted on inputs outside the 0–6 range (e.g., -1 or 7). The system correctly identifies these as "Out of Bounds" and re-prompts the user without losing the current game state.

7.2 Functional Logic Testing

To ensure the "Physics" of the game remained consistent, we performed **Black Box Testing** on the gravity-drop and win-detection engines.

- **Full Column Constraint:** A specific test case involved attempting to drop a piece into a column that already contained six pieces. The system successfully returned a "Column Full" warning and prevented the internal 2D list from exceeding its row dimensions.
- **Win Condition Coverage:** We manually simulated matches to verify all 4 win axes:

1. **Horizontal:** Four pieces in a single row.
2. **Vertical:** A stack of four pieces.
3. **Positive Diagonal:** A / shaped alignment.
4. **Negative Diagonal:** A \ shaped alignment.

- **Stalemate Recognition:** A "Full Board" test was conducted by filling all 42 slots without a 4-in-a-row. The system accurately identified the "Draw" state and terminated the loop.

7.3 AI Performance Benchmarking

The "Hard" AI's Minimax algorithm was subjected to **Performance Testing** to ensure it met the Non-Functional Requirement of low latency (under 2 seconds).

- **Node Count Analysis:** We monitored how many board states the AI evaluated with and without Alpha-Beta Pruning. Results showed that Pruning reduced the search space by approximately 65%, allowing for a depth of 5 moves to be calculated in roughly 0.4 seconds on a standard processor.
- **Strategic Verification (AI vs. AI):** To test the "Hard" difficulty's efficacy, we ran automated simulations of the "Hard" AI against the "Easy" (Random) AI. The Hard AI maintained a 100% win rate over 50 iterations, confirming that the heuristic scoring and Minimax logic were superior to stochastic movement.

7.4 Regression Testing

During the implementation of the AI, a common bug occurred where the AI would "see" a win that didn't exist or ignore a player's winning move. **Regression Testing** ensured that every time a new heuristic (like center-column preference) was added, the core win-detection logic remained untouched and accurate. This was achieved by re-running a set of "Standardized Board States" after every code modification.

---

8. CHALLENGES & SOLUTIONS

8.1 The "Ghost Move" Bug (Object Referencing)

One of the most significant technical hurdles encountered during the development of the AI was a phenomenon referred to as the "Ghost Move" bug. In Python, lists are objects passed by reference. Initially, when the AI attempted to simulate a future move, it was inadvertently modifying the live game board instead of a hypothetical one.

- **The Challenge:** The AI's recursive search would "drop" a piece into a column to evaluate the outcome, but because it was using the original board object, the piece would appear on the user’s screen before the AI even finalized its decision.

- **The Solution:** The implementation of the copy.deepcopy() method from Python's standard library. By creating a recursive clone of the board matrix for every branch of the Minimax tree, the AI was granted a "sandbox" to simulate millions of futures without corrupting the primary game state.

8.2 Index Out of Bounds (Boundary Logic)

Connect Four's win detection requires checking adjacent cells in four directions. Early iterations of the check_win function frequently crashed with IndexError exceptions when a piece was placed near the edges of the 6x7 grid.

- **The Challenge:** A win-check starting at the right-most column (Index 6) would attempt to look at Column 7, 8, and 9 to complete a horizontal set of four, exceeding the list dimensions.
- **The Solution:** The logic was refactored to include "Boundary Constraints." The loops were restricted so that a horizontal check only begins if the starting column is ≤ 3. Similarly, vertical checks only begin if the starting row is ≤ 2. This mathematical constraint ensures the algorithm never "looks" outside the physical boundaries of the board.

3. The "Horizon Effect" in Decision Making

During early testing of the "Hard" AI, the bot would occasionally make "suicidal" moves—ignoring a player's imminent win to pursue a long-term advantage that it couldn't actually reach. This is a classic AI problem known as the **Horizon Effect**.

- **The Challenge:** The AI's "vision" was limited to its search depth. If a player's win was 6 moves away and the AI only looked 5 moves ahead, it remained blissfully unaware of the danger until it was too late to block.
- **The Solution:** To counteract this, the **Heuristic Scoring System** was heavily weighted toward defense. By assigning a massive negative penalty (-80) to any window where the player has three pieces in a row, the AI was forced to prioritize blocking "threat sequences" even if it couldn't mathematically see the final "Game Over" state yet.

8.4 Terminal Flicker and UI Refreshing

Because the game runs in a command-line interface, every move originally resulted in a new board being printed below the old one, leading to a "wall of text" that was difficult to navigate.

- **The Challenge:** Creating a static, "app-like" feel in a scrolling terminal environment.
- **The Solution:** The integration of the os.system('cls' if os.name == 'nt' else 'clear') command. By clearing the terminal buffer immediately before redrawing the board, the UI appears to update in place, providing a much cleaner and more professional user experience across different operating systems.

---

9. CONCLUSION & FUTURE SCOPE

9.1 Summary of Achievements

The "First to 4" project successfully achieved its primary objective: the creation of a stable, high-performance, and strategically capable digital adaptation of Connect Four. By successfully integrating the **Minimax Algorithm** with **Alpha-Beta Pruning**, the project demonstrated that complex game theory can be efficiently executed within a Python environment without the need for high-end graphical engines.

Key milestones reached during development include:

- **Algorithmic Efficiency:** The AI can evaluate thousands of board states in under a second, providing a "Hard" difficulty that is mathematically challenging for even experienced human players.
- **Robust Game Logic:** The win-detection and gravity-drop engines were stress-tested to ensure zero logic errors or "out-of-bounds" crashes during extended gameplay sessions.
- **User Experience:** Despite being terminal-based, the use of ANSI color codes and screen-clearing functions provided a clean, modern interface that prioritizes game state clarity.

9.2 Critical Reflections

The development process highlighted the importance of **Heuristic Evaluation** in AI. While the Minimax algorithm provides the "skeleton" of the decision-making process, it is the heuristic scoring—valuing center columns and blocking 3-in-a-row threats—that gives the AI its "personality" and strategic effectiveness. This project served as a

practical application of data structures (2D lists) and recursive logic, reinforcing the principle that software quality is often determined by the elegance of its underlying mathematics.

9.3 Future Scope and Enhancements

While the current version is a complete and playable product, several avenues exist for future expansion to elevate the project to a professional-grade software suite.

9.3.1 Graphical User Interface (GUI) Integration

The current architecture follows a modular pattern, making it ready for a transition from the terminal to a window-based GUI. Utilizing libraries such as **Tkinter**, **PyQt**, or **Pygame** would allow for:

- Smooth animations of pieces "falling" into slots.
- Click-to-drop mouse interactions.
- A visually appealing menu system and difficulty slider.

9.3.2 Machine Learning and Reinforcement Learning

A significant advancement would be replacing the hard-coded heuristic scoring with a **Neural Network**. By using a technique called "Self-Play Reinforcement Learning" (similar to AlphaZero), the AI could play millions of games against itself to "learn" which board positions are strongest, rather than relying on human-defined scores for center columns or sequences.

9.3.3 Networked Multiplayer

The project currently supports Local Multiplayer (two players on one keyboard). A future iteration could implement **Socket Programming** to allow for "Remote Play." This would involve creating a Server-Client architecture where two players can compete over a local network or the internet, requiring the implementation of data serialization and latency handling.

9.3.4 Bitboard Optimization

To reach professional "engine" speeds, the board representation could be refactored from a 2D list into a **Bitboard** (representing the board as a 64-bit integer). This would allow win-checks and piece placements to be calculated using ultra-fast bitwise operations (AND,

OR, XOR, SHIFT), potentially allowing the AI to search to a depth of 12 or 15 moves instead of 5.